Trusted Devices in Banking Apps: Findings

Methods to implement

Banking apps layer several methods, because each one answers a different question: is this the same phone, is this the real app, and is this the real customer.

1. **Device identifier:**

The app creates a *random ID*, stores it in the Keychain with a "this device only" setting, and sends it with each request. The server keeps a list of known IDs per customer. It is easy to build, but anyone who captures a request can copy the ID.

2. **Device-bound key (device binding):**

At registration the app generates a private key inside the phone's secure hardware (Secure Enclave on iOS) and sends only the public key to the bank. Later sign ins and payments are signed with that key, which never leaves the device. Europe's banking regulator lists "a private key linking an app to a device" as valid proof of possession, while an app merely being installed is not (EBA SCA opinion, 2019).

3. **App and device integrity:**

App Attest on iOS lets Apple certify that a hardware key belongs to a genuine, unmodified copy of the app. Play Integrity does the same on Android and adds a device integrity verdict. Apple notes these keys do not survive reinstalling the app or restoring a backup, so a new key means a new registration.

4. **Step-up verification for new devices:**

A sign in from an unknown device needs an extra check, such as a one-time code or approval from an existing device. OWASP describes step-up as extra verification before sensitive actions, using a large bank transfer as its example (MASWE-0029).

5. **Biometrics tied to the key**

The device key or token is stored in the Keychain with an access control that requires Face ID, Touch ID or the device passcode before it can be read (Apple). The "current set" option makes the item unusable if someone enrols a new face or fingerprint.

6. **Device management and alerts**

The customer can see and remove trusted devices, and is alerted when one is added. NIST SP 800-63B says providers "SHALL notify the subscriber" when an authenticator is bound, through a channel separate from the binding itself, and must remove compromised authenticators promptly.

7. **Server-side risk signals**

The bank also weighs location, network, time and behaviour, and asks for step-up when something looks unusual, such as a new payee from a new device.

Comparison between methods

Device-bound keys plus integrity checks give the strongest proof, but they need a real device and a server that can verify signatures.

Method	Proves	Can it be copied?	Effort	Works in Simulator + Mockoon
Device identifier	Same app install	Yes, from any captured request	Low	Yes
Device-bound key (Secure Enclave)	Same physical phone	No, the key never leaves the hardware	Medium	No, needs a real iPhone and signature checks
App Attest / Play Integrity	Genuine app on genuine hardware	No	High	No, needs a real device and Apple or Google servers
Step-up code for new devices	Customer controls another channel	Codes can be phished	Low	Yes (mock code)
Biometrics tied to the key	The owner is holding the phone	No	Medium	Partly, Simulator has simulated Face ID
Device list + alerts	Customer can spot and revoke devices	Not applicable	Low	Yes (list); alerts need email or push

Method	Proves	Can it be copied?	Effort	Works in Simulator + Mockoon
Risk signals	Behaviour looks normal	Not applicable	High	No

How it can be applied to Vale

Vale can implements methods 1, 4 and 6, the ones that can be built and tested with the Simulator and Mockoon. Device-bound keys and App Attest are the production upgrade.

- **Device identifier:** `DeviceIdentity` saves a random `dev_<UUID>` in the Keychain (`ThisDeviceOnly`) and `APIClient` sends it as `X-Device-ID` on every request.
- **Step-up for new devices:** after the SMS code, `POST /auth/verify` returns 200 for a known device or 202 for a new one. A new device enters an email code and can choose "Trust this device".
- **Device management:** Profile lists trusted devices (`GET /devices`) and removes them (`DELETE /devices/:id`). Removing the current phone logs it out.

However, there are limitations:

- The device ID can be copied from a captured request, unlike a Secure Enclave key.
- Removing another device does not end a session already open on it; that needs server-side token checks.
- There are no new-device alerts, which NIST expects through a separate channel.
- Mockoon does not enforce that the email step follows a correct SMS code.

Next step for production: replace the device ID with a Secure Enclave key registered at sign in, sign sensitive requests with it, and add App Attest so the server knows the request came from the genuine app.

References

1. Apple Developer: https://developer.apple.com/documentation/devicecheck/establishing-your-app-s-integrity
2. Apple Developer: https://developer.apple.com/documentation/cryptokit/secureenclave

3. Apple Developer: https://developer.apple.com/documentation/localauthentication/accessing-keychain-items-with-face-id-or-touch-id
4. Android Developers: https://developer.android.com/google/play/integrity/overview
5. OWASP MAS: https://mas.owasp.org/MASWE/MASVS-AUTH/MASWE-0029/
6. NIST: https://pages.nist.gov/800-63-4/sp800-63b/events
7. Norton Rose Fulbright: https://www.nortonrosefulbright.com/de-de/wissen/publications/bf11898a/eba-delivers-opinion-clarifying-key-elements-of-strong-customer-authentication-sca-under-psd2